Lecture Notes: AlexNet

ImageNet Classification with Deep Convolutional Neural Networks

Deep Learning / AI Graduate Course

July 7, 2026

Abstract

These notes cover AlexNet (Krizhevsky, Sutskever, and Hinton, 2012), the convolutional neural network that won the ILSVRC-2012 ImageNet competition by a wide margin and catalyzed the modern deep learning era in computer vision. We derive the layer-by-layer architecture, the mathematics of convolution and pooling output sizes, the ReLU nonlinearity, Local Response Normalization, dropout regularization, the two-GPU training scheme, and the data augmentation strategy. We close with the historical significance of the result and connections to later architectures such as VGGNet.

Contents

1 Historical Context and Motivation **2**

2 Dataset and Task **2**

3 Architecture Overview **2**
3.1 Layer-by-Layer Specification . . . 3

4 Mathematics of the Convolutional Layers **4**
4.1 Parameter Counting . . . 4

5 Key Innovations **4**
5.1 ReLU Nonlinearity . . . 4
5.2 Local Response Normalization (LRN) . . . 5
5.3 Overlapping Max Pooling . . . 5
5.4 Dropout . . . 5
5.5 Data Augmentation . . . 5

6 Training Configuration **6**
6.1 Two-GPU Model Parallelism . . . 6

7 Results and Significance **6**

8 Bridge to VGGNet **7**

9 Exercises **7**

10 References **7**

1 Historical Context and Motivation

Before 2012, the dominant approach to image classification relied on hand-engineered features (SIFT, HOG, Fisher vectors) combined with a shallow classifier such as an SVM. Krizhevsky, Sutskever, and Hinton's submission to the ImageNet Large Scale Visual Recognition Challenge (ILSVRC-2012) reduced the top-5 error rate on a 1000-class, 1.2-million-image dataset from $\approx 26\%$ (best non-neural entry) to 15.3% — an almost unprecedented single-year improvement. This result is widely regarded as the event that reignited mainstream interest in deep learning.

Intuition: Why did it work in 2012 and not 2002?

Three ingredients converged: (1) large labeled datasets (ImageNet, $\sim$1.2M images), (2) GPU hardware capable of training networks with 60M parameters in a feasible amount of wall-clock time, and (3) algorithmic refinements — ReLU activations, dropout, and data augmentation — that made it practical to train a network deep and wide enough to exploit (1) and (2). None of these three alone was sufficient; the contribution of AlexNet was to combine them into a single working system at scale.

2 Dataset and Task

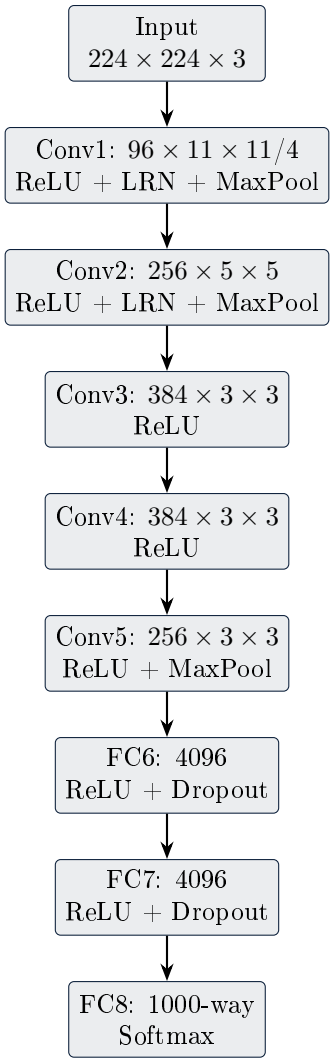
ILSVRC-2012 uses a subset of ImageNet with roughly 1.2 million training images, 50,000 validation images, and 150,000 test images spanning 1000 object categories. Images are variable resolution; AlexNet down-samples every image to a fixed 256×256 pixel size and then extracts (during training) random 224×224 crops.

Definition: Top-1 / Top-5 Error

For a prediction that outputs a ranked list of candidate labels, *top-1 error* counts a miss unless the highest-ranked label matches the ground truth. *Top-5 error* counts a miss only if the ground-truth label is absent from the five highest-ranked predictions. Top-5 error is the standard ILSVRC metric because ImageNet categories are fine-grained and often ambiguous (e.g., many dog breeds).

3 Architecture Overview

AlexNet consists of 8 learned layers: 5 convolutional layers followed by 3 fully connected layers, with the final layer feeding a 1000-way softmax. Max-pooling follows the 1st, 2nd, and 5th convolutional layers, and Local Response Normalization (LRN) follows the 1st and 2nd convolutional layers.



3.1 Layer-by-Layer Specification

Layer	Kernel	Stride	Output shape	Params	Notes
Input	–	–	$224 \times 224 \times 3$	–	
Conv1	11×11, 96	4	$55 \times 55 \times 96$	34,944	ReLU, LRN
Pool1	3×3	2	$27 \times 27 \times 96$	0	overlapping max-pool
Conv2	5×5, 256	1	$27 \times 27 \times 256$	614,656	pad 2, ReLU, LRN
Pool2	3×3	2	$13 \times 13 \times 256$	0	overlapping max-pool
Conv3	3×3, 384	1	$13 \times 13 \times 384$	885,120	pad 1, ReLU
Conv4	3×3, 384	1	$13 \times 13 \times 384$	1,327,488	pad 1, ReLU
Conv5	3×3, 256	1	$13 \times 13 \times 256$	884,992	pad 1, ReLU
Pool5	3×3	2	$6 \times 6 \times 256$	0	overlapping max-pool
FC6	–	–	4096	37,748,736	ReLU, Dropout 0.5
FC7	–	–	4096	16,777,216	ReLU, Dropout 0.5
FC8	–	–	1000	4,096,000	Softmax

The total parameter count is approximately 62 million, of which nearly 95% reside in the three fully

connected layers — a ratio that motivated later architectures (e.g., GoogLeNet, and eventually global average pooling designs) to reduce the fully connected footprint.

4 Mathematics of the Convolutional Layers

Definition: Output Spatial Size of a Convolution

For an input of spatial size W, a kernel of size F, stride S, and zero-padding P on each side, the output size along that dimension is

$$W_{\text{out}} = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1.$$

Worked Example: Conv1 output size

Input 224×224, kernel $F = 11$, stride $S = 4$, padding $P = 0$ (as in the original paper; some re-implementations use $P = 2$ to hit exactly 55×55 from 227×227 input):

$$W_{\text{out}} = \left\lfloor \frac{224 - 11}{4} \right\rfloor + 1 = \lfloor 53.25 \rfloor + 1 = 54.$$

Because 224 does not divide evenly, the original paper's 55×55 figure is obtained by treating the true input as 227×227:

$$\left\lfloor \frac{227 - 11}{4} \right\rfloor + 1 = \lfloor 54 \rfloor + 1 = 55.$$

This discrepancy between the paper's stated 224×224 input and the arithmetic that actually produces 55×55 is a well-known erratum; when implementing AlexNet from scratch, use 227×227 input (or equivalent padding) to match the published feature-map sizes exactly.

4.1 Parameter Counting

For a convolutional layer with C_{in} input channels, C_{out} output channels, and kernel size $F \times F$, the number of learnable parameters (including one bias per output channel) is

$$\text{Params} = C_{\text{out}} \left(C_{\text{in}} \cdot F^2 + 1 \right).$$

For Conv2 ($C_{\text{in}} = 96$, $C_{\text{out}} = 256$, $F = 5$): $256\,(96 \cdot 25 + 1) = 256 \cdot 2401 = 614{,}656$, matching the table above.

5 Key Innovations

5.1 ReLU Nonlinearity

AlexNet popularized the Rectified Linear Unit,

$$f(x) = \max(0, x),$$

in place of the saturating tanh or sigmoid nonlinearities used previously. Because $f'(x) \in \{0, 1\}$ for $x \neq 0$, gradients do not vanish for large positive activations, which the original paper reports yields roughly a 6× speedup in reaching a target training error on CIFAR-10-scale networks.

Intuition: Why ReLU trains faster

tanh and sigmoid squash their input into a bounded range and have derivatives that approach zero away from the origin — a unit that is "saturated" barely updates during backpropagation. ReLU's derivative is a constant 1 for all positive inputs, so gradient magnitude is preserved through many stacked layers, at the cost of "dead units" that receive only negative input and stop learning entirely.

5.2 Local Response Normalization (LRN)

LRN implements a form of lateral inhibition across feature channels at the same spatial location. For activity $a^i_{x,y}$ of kernel i at position (x, y), the normalized response is

$$b^i_{x,y} = \frac{a^i_{x,y}}{\left(k + \alpha \sum_{j=\max(0,\, i-n/2)}^{\min(N-1,\, i+n/2)} \left(a^j_{x,y}\right)^2\right)^{\beta}},$$

with the original paper using $k = 2$, $n = 5$, $\alpha = 10^{-4}$, $\beta = 0.75$. LRN reduced top-5 error by about 1.4% in the original ablations but has since fallen out of favor; Batch Normalization (Ioffe & Szegedy, 2015) largely superseded it and is the modern default.

5.3 Overlapping Max Pooling

Pooling windows of size 3×3 with stride 2 overlap (stride $<$ kernel size), which the authors found reduced top-1/top-5 error by 0.4%/0.3% relative to the traditional non-overlapping (2×2, stride 2) scheme, while also modestly reducing overfitting.

5.4 Dropout

Dropout (Srivastava et al., 2014, used here in one of its earliest large-scale applications) randomly zeroes each hidden unit's output with probability $p = 0.5$ during training, forcing the network to learn redundant, non-co-adapted representations. At test time all units are retained but their outputs are scaled by $(1-p)$ — equivalently, this is an approximate model average over 2^n "thinned" sub-networks sharing weights. AlexNet applies dropout only to FC6 and FC7; without it, the authors report substantial overfitting given only ∼1.2M training images against 62M parameters.

5.5 Data Augmentation

Two low-cost augmentation schemes are used to combat overfitting without materially increasing training time:

- **Random crops and horizontal flips**: extracting random 224×224 patches (and their horizontal reflections) from the 256×256 images, increasing the effective dataset size by a factor of 2048 (and averaging 10 crops — 4 corners + center, each flipped — at test time).

- **PCA color augmentation**: adding multiples of the principal components of RGB pixel values across the training set,

$$\Delta I_{xy} = [\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3]\,[\alpha_1\lambda_1, \alpha_2\lambda_2, \alpha_3\lambda_3]^\top, \qquad \alpha_i \sim \mathcal{N}(0, 0.1^2),$$

where $\mathbf{p}_i$ and λ_i are the eigenvectors and eigenvalues of the 3×3 covariance matrix of RGB pixel values. This reduced top-1 error by over 1%.

6 Training Configuration

Hyperparameter	Value
Optimizer	SGD with momentum 0.9
Weight decay	5×10^{-4} (acts as a regularizer, not merely decay)
Batch size	128
Learning rate	0.01, divided by 10 when validation error plateaus (3 times total)
Weight initialization	$\mathcal{N}(0, 0.01^2)$; bias $= 1$ for Conv2/4/5 and FC layers, bias $= 0$ elsewhere
Epochs	$\sim$90 passes over the training set
Hardware	2$\times$ NVIDIA GTX 580 (3GB each), $\sim$5–6 days training

The weight update rule used is

$$v_{t+1} = 0.9\,v_t - 0.0005\,\epsilon\,w_t - \epsilon \left\langle \frac{\partial L}{\partial w}\Big|_{w_t} \right\rangle_{D_i}, \qquad w_{t+1} = w_t + v_{t+1},$$

where ϵ is the learning rate and the gradient is averaged over the current batch D_i.

6.1 Two-GPU Model Parallelism

Because a single GTX 580 had only 3GB of memory, the network is split across two GPUs: each GPU holds half of the kernels (feature maps) in every layer, and the two streams communicate only at specific layers (Conv3 and all FC layers connect across both GPUs; Conv2, Conv4, Conv5 connect only within the same GPU). This is a historical/engineering artifact of 2012 hardware limits rather than a fundamental architectural principle, but it foreshadows model-parallel training strategies used in much larger networks today.

7 Results and Significance

Model	Top-1 error	Top-5 error
Best non-CNN (Fisher Vectors + SVM), ILSVRC-2012	–	26.2%
AlexNet (single network)	40.7%	18.2%
AlexNet (7-CNN ensemble)	36.7%	15.3%

Why AlexNet matters

AlexNet was not the first CNN (LeNet-5 dates to 1998) nor did it introduce a fundamentally new layer type. Its contribution was demonstrating, at unprecedented scale, that (a) depth plus (b) GPU-accelerated training plus (c) ReLU/dropout/augmentation could dramatically

outperform hand-engineered feature pipelines — and that this held on a dataset large and diverse enough (ImageNet) to be broadly convincing. It set the template — conv/pool stacks feeding fully connected classifier layers, trained end-to-end with SGD — that essentially every subsequent vision architecture (ZFNet, VGGNet, GoogLeNet, ResNet) refined rather than replaced.

8 Bridge to VGGNet

The companion lecture on VGGNet (Simonyan & Zisserman, 2014) can be understood as asking: *what happens if we make the network deeper while keeping every convolutional filter small (3×3)?* AlexNet's use of large 11×11 and 5×5 filters in early layers is exactly the design choice VGGNet revisits — replacing them with stacks of 3×3 convolutions that achieve a larger effective receptive field with fewer parameters and more nonlinearities. Keep the parameter-counting formula from Section 4 handy; it is reused directly in the VGGNet notes.

9 Exercises

1. Verify the output spatial size of Pool5 given a Conv5 output of $13 \times 13 \times 256$, kernel 3×3, stride 2, no padding.
2. Compute the number of parameters in FC6, given that Pool5's output is flattened to a vector of length $6 \times 6 \times 256 = 9216$ and FC6 has 4096 units.
3. Derive the effective dataset size multiplier from random cropping ($256 \times 256 \to 224 \times 224$) and horizontal flipping. (Hint: count the number of distinct top-left crop offsets, then multiply by 2 for flips.)
4. Explain, in terms of the bias-variance tradeoff, why dropout was applied only to the fully connected layers and not the convolutional layers in the original design.
5. Compare the total parameter count of AlexNet's fully connected layers to its convolutional layers. What design principle in later architectures (e.g., global average pooling in GoogLeNet/ResNet) directly addresses the imbalance you find?

10 References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

[2] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, 15(1), 2014.

[3] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," *ICML*, 2015.

[4] O. Russakovsky et al., "ImageNet Large Scale Visual Recognition Challenge," *International Journal of Computer Vision*, 115(3), 2015.